

Exponentiation rapide

0.1 Principe

L'exponentiation rapide repose sur les identités suivantes :

$$a^n = \begin{cases} 1 & \text{si } n = 0, \\ (a^{n/2})^2 & \text{si } n \text{ est pair,} \\ (a^{(n-1)/2})^2 \cdot a & \text{si } n \text{ est impair.} \end{cases}$$

À chaque appel récursif, l'exposant est divisé par deux, ce qui permet de réduire fortement le nombre de multiplications nécessaires par rapport à la méthode naïve.

0.2 Complexité

Cas	Complexité
Meilleur cas	$\Theta(\log n)$
Pire cas	$\Theta(\log n)$
Cas moyen	$\Theta(\log n)$

Remarque : Pour le démontrer : par récurrence forte sur $n \geq 1$, on montre que pour tout $n \geq 1$, le nombre total de multiplications effectuées par un appel à `expo_rap(a, n)` est inférieur ou égal à $2 + 2\lfloor \log_2 n \rfloor$.

0.3 Correction

Proposition : soit $k \in \mathbb{N}$.

$$H_k : \quad \forall a \in \mathbb{R}, \quad \text{expo_rec}(a, k) \text{ termine et renvoie } a^k.$$

Démonstration par récurrence forte sur k .

Initialisation.

L'appel de `expo_rec(a, 0)` renvoie $1 = a^0$ et termine par condition d'arrêt des appels récursifs. Donc H_0 est vraie.

Hérédité.

Soit $k \geq 1$. On suppose que pour tout $j \in [0, k-1]$, H_j est vraie, c'est-à-dire

$$\forall a \in \mathbb{R}, \quad \text{expo_rec}(a, j) \text{ termine et renvoie } a^j.$$

On raisonne suivant la parité de k .

1. Cas où k est pair.

Il existe $j \in [0, k - 1]$ tel que $k = 2j$.

Par hypothèse de récurrence, l'appel de `expo_rec(a,j)` termine et renvoie

$$a^j = a^{k/2}.$$

Ainsi, `expo_rec(a,k)` renvoie

$$a^j \times a^j = a^{k/2} \times a^{k/2} = a^k.$$

De plus, l'exécution de `expo_rec(a,k)` ne comporte que des instructions de coût constant en plus de l'appel `expo_rec(a,j)`, lequel termine ; donc `expo_rec(a,k)` termine.

2. Cas où k est impair.

Il existe $j \in [0, k - 1]$ tel que $k = 2j + 1$.

Par hypothèse de récurrence, l'appel de `expo_rec(a,j)` termine et renvoie

$$a^j.$$

Ainsi, `expo_rec(a,k)` renvoie

$$a \times a^j \times a^j = a^{2j+1} = a^k.$$

Comme précédemment, l'appel `expo_rec(a,k)` ne comporte qu'un nombre constant d'opérations supplémentaires après l'appel récursif `expo_rec(a,j)`, qui termine ; donc `expo_rec(a,k)` termine.

On a donc établi H_k . Par récurrence forte, pour tout $k \in \mathbb{N}$, `expo_rec(a,k)` termine et renvoie a^k .

La post-condition est donc vérifiée (correction partielle), et la terminaison ayant été démontrée, la **correction totale** de l'algorithme est établie.

—

0.4 Pseudo-code (récursif)

Exponentiation rapide

expo_rap (a, n)

Entrée : $a \in \mathbb{R}, n \in \mathbb{N}$

Sortie : $p = a^n$

si $n \leq 0$ alors

retourner 1

finsi

si $n \bmod(2) = 0$ alors

$b := \text{expo_rap}(a, n/2)$

retourner $b * b$

sinon

$b := \text{expo_rap}(a, (n-1)/2)$

retourner $b * b * a$

finsi

1 Implémentation en C

Tri par sélection

```
int expo_rap_iter(int a, int n) {
    int p = 1;
    int x = a;

    while (n > 0) {
        if (n % 2 == 1)
            p = p * x;
        x = x * x;
        n = n / 2;
    }
    return p;
}
```

On remarque les mêmes calculs :

p joue le rôle de la valeur accumulée du résultat ($b*b$ ou $b*b*a$ dans le récursif).

x stocke $a^{(2^k)}$ à chaque itération, comme les appels récursifs divisant l'exposant par 2.

La mise à jour de n simule la réduction de l'exposant dans les appels récursifs.

2 Implémentation en Caml

Exponentiation rapide

```
let rec expo_rap (a : int)(n : int) : int =  
  if n = 0 then 1  
  else if n mod 2 = 0 then  
    let b = expo_rap a (n/2)  
    in b*b  
  else  
    let b = expo_rap a ((n-1)/2)  
    in b*b*a
```

C'est ici une implémentation non terminale et fidèle au pseudo code.